

GSoC 2026 Proposal

Optimize a VLA Model for Embodied Intelligence: OpenVINO Toolkit (Intel) Project #21

About Me

Full name: Mohamed Deraz Nasr

University: Georgia Institute of Technology, MS Computer Science (Machine Learning Track)

Timezone: Eastern Time (ET, UTC-4)

Contact: mderaznasr@gmail.com | <https://github.com/MDerazNasr/>

Bio

I'm currently pursuing a Masters in CS at Georgia Tech studying machine learning. My goal is to do research in world models and physical AI. To move towards that, I am building experience through ML research and personal projects. Right now I am working on a world model for robot manipulation and an inference engine for flow matching policies. I also have work experience spanning different aspects of ML, most notably my last internship at Shopify where I built ML pipelines and worked on a Pytorch LLM to convert English to SQL.

Programming experience:

- C++ (4 years), Python (5 years), CUDA (2 years)
- PyTorch, HuggingFace Transformers, ONNX Runtime, TensorRT
- **Systems experience:** C++/CUDA inference engine, SE(3)-equivariant CUDA kernels for protein generation, distributed CUDA MCTS with async PyTorch weight sync (ongoing at Georgia Tech).
- **Production ML:** Shopify ML Engineering internship - Built ML pipelines in Python, Spark, SQL, and Airflow; reduced real-time XGBoost prediction latency by 35%; prototyped a PyTorch LLM to convert natural language to SQL.
- **Embedded/RT:** STM32/FreeRTOS firmware for exoskeleton motor control (UOBionics), contributed frequency interpolation layer to ros2_control ONNX locomotion demo.

Relevant technology experience

- **Flow matching and inference optimization:** I am working on a C++/CUDA inference engine targeting Diffusion Policy and FLUX.1. It uses persistent trajectory CUDA kernels, INT8 quantization via CUTLASS, and ONNX Runtime as the model backend. I extended it to support the VLM + flow matching DiT architecture used in current VLA models, resolving the same four export blockers described in this proposal.
- **Model export and deployment:** I have exported models to ONNX and TensorRT and debugged export failures at the op level.
- **Intel GPU and oneDNN:** I have read the OpenVINO GPU plugin source: the oneDNN primitive registration, the kernel selector, and the fusion passes in graph_optimizer/ that

control whether ops like LayerNorm and attention are fused or decomposed. I am familiar with VTune GPU Hotspots profiling and will use it to identify bottlenecks before writing any kernel code.

Experience

Open Source Contributions

ros2_control_demos: Pre-application contribution, [PR](#), [Code](#) (March 2026)

- Built on PR #1053 (Open Duck Mini ONNX locomotion demo) by adding a frequency interpolation layer to the motion controller.
- The controller computes $N = \text{round}(T_{\text{training}} / T_{\text{control}})$ on the first active update, runs ONNX inference every N ticks, and linearly interpolates joint commands between outputs.
- This decouples inference rate from control rate with no configuration required.
- Also added a headless Dockerfile using OSMesa software rendering so the demo runs in CI without a display.

openvinotoolkit/opencvino: Pre-application contribution, [PR #35027](#) (March 2026)

- Added MVN eltwise fusion tests for the AdaLayerNorm pattern which the IR subgraph that the Weeks 7 to 9 (discussed below) kernel contribution targets. Covers both static and dynamic scale and shift inputs, filling a gap in the existing MVN fusion test suite.

Research and Industry Experience

Georgia Institute of Technology, Machine Learning Researcher, supervised by Dr. Adibi (Jan 2026 to present)

- Designing a distributed CUDA MCTS system with multi-node Ray workers and async inference weight sync via PyTorch.
- Developing MA-UCB, a bandit algorithm with formal regret bounds for directing compute toward missing state variables.
- Building a multi-encoder pipeline that fuses sensor streams into a semantic state space for planning over incomplete observations.

Shopify, Machine Learning Engineer Intern (Jan 2025 to Apr 2025)

- Built ML pipelines in Python, Spark, SQL, and Airflow to forecast business KPI metrics.
- Reduced real-time XGBoost prediction latency by 35% using Redis and Docker.
- Prototyped a PyTorch LLM to convert natural language to SQL, cutting query time by 80% on tables with over 1 million records.

University of Ottawa, Research Engineer Intern, supervised by Dr. Kalonji (Sep 2024 to Dec 2024)

- Deployed a reinforcement learning retraining controller with EWMA drift detection, reducing recompute by 40%.
- Built data pipelines in Python and C++ to process over 200K solar readings per day.
- Built a Next.js and GraphQL dashboard with XGBoost forecasts updated at under 200 ms latency.

March Networks, Systems Engineer Intern (Jan 2023 to Apr 2023)

- Built a C/C++ diagnostic tool to debug RTSP and WebSocket failures on Linux, improving detection rate by 25%.
- Developed a Node.js and PostgreSQL dashboard to track packet loss across 200 deployed camera networks.

UOBionics, Embedded Developer, Allonstride Exoskeleton (Jan 2024 to Dec 2024)

- Wrote STM32/FreeRTOS C/C++ firmware for exoskeleton motor control.
- Built an AWS pipeline for live anomaly detection on over 100K signals.

Projects

FlowRT (Jan 2026 to present)

- An in-progress C++/CUDA inference engine for flow matching models targeting Diffusion Policy and FLUX.1.
- Implements persistent trajectory CUDA kernels and INT8 quantization via CUTLASS.
- Targeting 10x lower latency than a standard PyTorch setup.
- Uses ONNX Runtime as the model backend, the same runtime used in `ros2_control_demos` example_18.

Protein Diffusion (Oct 2025 to Feb 2026)

- Built an SE(3)-equivariant DDPM that generates protein backbones in around 10 seconds with AlphaFold plausibility scoring, inspired by RFdiffusion.

Reinforcement Learning F1 Race Simulator (Aug 2025 - Oct 2025)

- Designed F1 strategy sim with PPO & behavioral cloning; surpassing expert by 56% in total race reward & 2x baseline lap speed.

Publications

Affinity Map: Few-Shot Protein Family Classification via Prototypical Networks (Zenodo, 2026)

- Published a meta-learning study achieving 89% 5-shot accuracy on unseen protein families using ESM-2 and LoRA fine-tuning.

About the Project

My choice: Optimize a VLA Model for Embodied Intelligence (Project #21)

Why I chose this project:

I am pursuing a Masters in CS at Georgia Tech studying machine learning. My goal is to do research in physical AI and robot learning. To move toward that, I am building experience through ML research and personal projects. Right now I am working on a world model for robot manipulation and an inference engine for flow matching policies.

Time investment:

I will invest 30 to 35 hours per week across the 13-week coding period, for a total of approximately 350 hours. I have no other internship or major commitment this summer so GSoC is my primary focus.

Abstract

Vision-Language-Action (VLA) models like unifolm-vla combine a vision-language model (VLM) backbone with a flow matching action head. The action head runs a denoising loop of N steps per action chunk, where each step is a full forward pass through a cross-attention DiT transformer. On Intel iGPU, two problems prevent real-time deployment.

First, the model cannot run in OpenVINO at all. I analyzed the unifolm-vla source code and found four concrete export blockers in `openvino.convert_model`. These must be resolved before any optimization is done.

Second, even after export, the denoising loop will not reach control frequency without kernel level work. The cross-attention in the DiT must dispatch to the fused SDPA kernel. The AdaLayerNorm pattern (LayerNorm + affine scale/shift conditioned on a timestep embedding) has no fused op in the GPU plugin today. Implementing it as a fused operation is a primary goal.

The project has three deliverables:

1. A working OpenVINO export pipeline for unifolm-vla, with all blockers resolved and both model stages running on Arrow Lake or Panther Lake iGPU.
2. Kernel-level optimizations targeting bottlenecks identified by VTune profiling.
3. A reference project for unifolm-vla: export scripts, profiling report, quantization configs, and an end-to-end latency benchmark.

As an extra goal, if time permits, I will implement a RobotActionPipeline C++ class in OpenVINO GenAI that moves the Euler denoising loop out of Python and into C++.

Detailed Timeline

Community Bonding Period (May 1 to May 25)

- Read the ImageGenerationPipeline and LLMPipeline C++ implementations in full.
- Work through the oneDNN developer guide. Confirm iGPU hardware access with mentors.
- Run the unifolm-vla PyTorch baseline on CPU.

Weeks 1 to 2 (May 26 to June 8): Export Blockers

- **Blocker 1:** Python for-loop denoising (primary structural blocker). File: DiT_ActionHeader.py, line 251. Problem: The denoising loop calls the full DiT N times inside a Python for-loop. Tracing this unrolls it into N copies of the full DiT graph in a single static IR. Fix: Export the DiT as a single-step model.
- **Blocker 2:** torch.randn inside the inference path. File: DiT_ActionHeader.py, line 239. Fix: Export wrapper that accepts the noise tensor as an external input.
- **Blocker 3:** torch.autocast("cuda") in the forward pass. File: unifolm_vla.py, lines 46, 85, 98. Fix: Remove autocast from the forward path and set precision at OpenVINO compile time via ov_config={"INFERENCE_PRECISION_HINT": "bf16"}.
- **Blocker 4:** BatchFeature API boundary. Fix: Thin tensor-in / tensor-out wrapper for each model stage.
- **Deliverable:** both models exported to OpenVINO IR and numerically validated against PyTorch outputs on CPU.

Weeks 3 to 4 (June 9 to June 22): iGPU Baseline

- Compile both models for GPU on Arrow Lake or Panther Lake (Intel DevCloud if hardware is not available in time).
- Measure backbone prefill latency, DiT single-step latency, and total action chunk latency.
- **Deliverable:** documented baseline latency numbers.

Weeks 5 to 6 (June 23 to July 6): VTune Profiling

- Run VTune GPU Hotspots and check primitive selection with DNNL_VERBOSE=1.
- Confirm cross-attention hits the fused SDPA kernel and AdaLayerNorm is not running as three separate ops.
- **Deliverable:** profiling report with top bottlenecks identified.

Weeks 7 to 9 (July 7 to July 27): Kernel Contributions

- AdaLayerNorm fusion: fuse LayerNorm + timestep-conditioned scale/shift into a single op.
- SDPA fix: if cross-attention is not hitting the fused path, fix the pattern match.
- **Deliverable:** at least one merged or submitted PR to openvino/toolkit/openvino.

Weeks 10 to 11 (July 28 to August 10): Quantization

- INT4 weight-only quantization on Qwen2.5-VL via NNCF.
- INT8 on the DiT if profiling showed it is compute-bound.
- Accuracy validated via MAE.
- **Deliverable:** quantization configs and updated latency measurements.

Weeks 12 to 13 (August 11 to August 17): Benchmarking and Documentation

- Full end-to-end benchmark on iGPU.
- Write portability documentation: how to export any VLA model with a VLM + flow matching DiT structure to OpenVINO IR.
- The four export blockers and the AdaLayerNorm fusion apply directly to GR00T N1.5 and Pi0, which share the same DiT + flow matching action head architecture.
- Stretch goal: RobotActionPipeline C++ class in OpenVINO GenAI.
- **Deliverable:** benchmark results, portability documentation, and final report.

Final Evaluation: August 17 to 24

General Questions

How do you know OpenVINO?

I found OpenVINO while looking into options for my robot learning projects. I specifically wanted to see if a Diffusion Policy could achieve real-time control frequencies on an Intel iGPU without requiring a discrete GPU.

What do you know about OpenVINO?

OpenVINO is Intel's toolkit for optimizing and deploying AI models on their CPUs, iGPUs, and NPUs. It works by taking a model from PyTorch and converting it into an "Intermediate Representation," which the engine then compiles into instructions specific to the hardware you're using. For iGPUs, it maps the math to oneDNN primitives or OpenCL kernels tuned for the Xe architecture. It's useful for robotics because of graph fusing and the GenAI library, which helps run iterative tasks like Diffusion Policies fast enough for real-time control.

Have you already contributed to OpenVINO?

Yes. I ran the existing GenAI pipeline examples locally and traced through the GPU plugin source to understand how ops move from IR to kernel dispatch. I opened a discussion on [openvinotoolkit/openvino.genai](#) with the export gap analysis for the target model, which the mentor reviewed and responded to. I also submitted [PR #35027](#) to [openvinotoolkit/openvino](#) adding MVN eltwise fusion tests for the AdaLayerNorm pattern, which is the exact IR subgraph that the Weeks 7 to 9 kernel contribution targets.

Professional development:

My goal is to do research in physical AI and robot learning. To get there I need to understand not just how models are trained, but how they run on real hardware. My current work is entirely

on CUDA and NVIDIA. Contributing kernel-level work to OpenVINO would give me direct experience with a production inference compiler on a different hardware backend.

Why should we pick you?

I'm currently pursuing a Masters in CS at Georgia Tech studying machine learning. My goal is to do research in top AI labs in world models and physical AI. To move toward that, I am building experience through ML research and personal projects. Right now I am working on a world model for robot manipulation and an inference engine for flow matching policies ([FlowRT](#)). Contributing to this project would give me direct experience with a production inference compiler on a different hardware backend, which is a gap in my current work that I need to close to do serious research in physical AI.

My background is a perfect fit for this project, as I have built a CUDA inference engine ([FlowRT](#)), debugged ONNX export failures at the op level, and I am doing active research in distributed CUDA inference at Georgia Tech. Before writing this proposal, I traced the target model, found the four blockers with file names and line numbers, and extended FlowRT to verify the fixes work. The denoising loop latency problem is not new to me as I encountered it on CUDA, built a solution for it, and would definitely be able to solve it on a different hardware backend.

Furthermore, I have lots of real experience shipping ML systems in a production environment at Shopify: code review, deadlines, async collaboration. I know how to move work through a team and how to take feedback on a PR.

I also do my Masters remotely. I manage my own schedule, set my own deadlines, and deliver research deliverables without any supervision. I will take ownership and responsibility of any work assigned to me and manage my time effectively.

Prerequisites

I opened a GitHub discussion on `openvinotoolkit/openvino.genai` with the full export gap analysis prior to submission, which mentor (baihe-liu) reviewed and responded to in that same thread. I also submitted [PR #35027](#) to `openvinotoolkit/openvino` adding MVN eltwise fusion tests for the AdaLayerNorm pattern which is the exact IR subgraph that the Weeks 7 to 9 kernel contribution targets.